

BIMGUARD AI Development Roadmap

Executive Summary

BIMGUARD AI is a unified compliance engine for Building Information Modeling (BIM) that automates validation of BIM Execution Plans (BEPs) and spatial building codes. This roadmap outlines a 24-month development plan structured across four phases, progressing from foundational capabilities to enterprise-scale deployment. The system addresses the critical gap between unstructured regulatory documents and structured IFC model validation, combining Large Language Models (LLMs), computational geometry, and semantic web technologies to transform manual compliance checking into automated, AI-driven validation.

Project Team: Letícia Cristovam Clemente, Malak Yaseen, Marc Azzam, Mark Shane Haines, Osama Ata

Target Market: Architecture, Engineering, Construction, and Operations (AECO) industry, addressing \$15.8 billion in annual losses from inadequate interoperability[1]

Phase 1: Foundation and MVP (Months 1-6)

Objective

Establish core infrastructure and demonstrate proof-of-concept for the three foundational pillars: IFC parsing, rule-based validation, and BCF output generation.

Key Deliverables

1.1 IFC Data Pipeline (Months 1-2) Goal: Reliable extraction and normalization of IFC model data

Technical Implementation:

- Deploy IfcOpenShell library for IFC4 Reference View and IFC2x3 Coordination View parsing
- Develop data extraction pipeline converting IFC entities to structured pandas dataframes
- Build spatial hierarchy indexing (building → storey → space → element relationships)
- Create property set extraction module for custom attributes
- Implement geometric feature extraction (bounding boxes, centroids, volumes)

Code Standard and Rules Implemented and Priority Requirements
IBC 2024 Fire Safety and 25 rules and Exit width, fire ratings, clearances
ADA Accessibility and 18 rules and Door width, turning radius, slopes
NFPA 70 Electrical and 12 rules and Panel clearances, working space

Table 1: Phase 1 rule coverage targets

Success Criteria:

- Parse 95% of buildingSMART Sample-Test-Files repository successfully[2]
- Extract geometry and metadata for 100,000+ element models within 30 seconds
- Generate element catalogs, adjacency tables, and equipment inventories

Key Technologies: Python, IfcOpenShell, pandas, numpy

1.2 Hard-Coded Rule Engine (Months 2-4) Goal: Implement deterministic validation for five priority element types

Target Elements:

1. Doors (fire exits, accessible entries, standard)
2. Windows (egress, ventilation, fire-rated)
3. Ducts (clearance zones, fire dampers)
4. Structural beams (load paths, clearances)
5. Stairs (rise/run ratios, handrails, egress width)

Rule Coverage:

Technical Implementation:

- Python-based rule evaluation engine with JSON rule definitions
- Bounding box-based spatial clash detection (AABB intersection tests)
- Violation tracking with element references and rule citations
- Processing target: <50ms per rule evaluation

Success Criteria:

- 100% accuracy on deterministic rule checks (door width, fire rating presence)
- Detection of 90%+ spatial clearance violations using bounding box approximations
- Validation report generation in <5 seconds for 10,000-element models

1.3 BCF Output Generation (Month 4) Goal: Generate industry-standard BIM Collaboration Format reports

Technical Implementation:

- BCF 2.1 XML schema implementation
- Viewpoint generation with camera positions and element highlighting
- Issue metadata: severity, rule reference, responsible discipline
- GUID-based element referencing compatible with Revit, Navisworks, Solibri

Success Criteria:

- BCF files importable into Autodesk Revit without errors
- Correct viewpoint navigation to violation locations
- Issue descriptions include element IDs, rule text, and recommended remediation

1.4 Command-Line Interface (Months 5-6) Goal: Deliver functional MVP for pilot testing

Features:

- IFC model upload and validation job submission
- Rule set selection (IBC + ADA + local code combination)
- Validation execution with progress reporting
- BCF report download
- Text-based violation summary

Deployment: Docker container with CLI interface for internal testing

Success Criteria:

- Complete validation workflow operational end-to-end
- Pilot testing with 3 internal IFC models demonstrating violation detection
- Documentation: API reference, rule coverage matrix, sample reports

Phase 1 Milestones

Month	Milestone	Deliverable
2	IFC Pipeline Complete	Parse 10 sample models successfully
4	Rule Engine Functional	Validate 55 hard-coded rules
5	BCF Generation Working	Import test report into Revit
6	MVP Release	CLI tool ready for pilot program

Phase 2: Semantic Enrichment and Intelligence (Months 7-12)

Objective

Introduce AI-driven semantic classification to bridge the gap between element form (geometry) and function (operational role), enabling context-aware compliance checking.

Key Deliverables

2.1 Graph Neural Network Development (Months 7-9) Goal: Classify ambiguous building elements using spatial and topological context

Problem Statement:

IFC models store geometric form (IfcDoor, width=0.9m) but lack functional classification (is it a fire exit, accessible entry, or closet door?). Rule requirements vary dramatically by function[3].

GNN Architecture:

- Edge-Conditioned Graph Neural Network (EC-GNN) targeting 91
- Node features: geometry (dimensions, aspect ratios, material properties)
- Edge features: spatial relationships (adjacency, distance, topological connectivity)
- Training approach: Transfer learning from pre-trained PointNet++ on 3D objects
- Fine-tuning dataset: 500-1,000 labeled building elements from historic projects

Implementation Steps:

1. Construct spatial graph from IFC model (elements as nodes, adjacency as edges)
2. Extract geometric point clouds and material properties per element
3. Train GNN classifier for 8 priority categories:
 - Fire exit doors vs standard doors
 - Egress corridors vs standard corridors
 - Accessible routes vs general circulation
 - Maintenance equipment vs general mechanical

4. Deploy inference pipeline with confidence thresholding (>0.8 = auto-classify, 0.6-0.8 = human review, <0.6 = flag as ambiguous)

Success Criteria:

- 85%+ classification accuracy on test set
- Inference time <200ms per element (parallelizable across 8 CPU cores)
- Confidence scores provided for all predictions enabling human oversight

Key Technologies: PyTorch Geometric, PyTorch, IfcOpenShell (geometry extraction)

2.2 RDF Knowledge Graph Construction (Months 8-10) Goal: Create semantic representation of BIM model for advanced querying and reasoning

Technical Implementation:

- Convert IFC elements to RDF triples using ifcOWL ontology[5]
- Enrich with GNN-inferred functional classifications
- Store in Apache Jena triple store for SPARQL querying
- Link to external ontologies: buildingSMART Data Dictionary, local code taxonomies

RDF Schema Example:

```
ex:Element_Door_001 rdf:type ifc:IfcDoor ; rdf:type ex:FireExit ; # GNN inference
ex:width "1.05"^^xsd:decimal ; ex:height "2.13"^^xsd:decimal ; ex:adjacentTo
ex:Stairwell_234 ; ex:inferenceConfidence "0.94"^^xsd:decimal ; ex:fireRating
"90-min"^^xsd:string .
```

Spatial Query Capabilities:

- GeoSPARQL for geometric relationships (within, intersects, adjacent)
- Path finding for egress route validation
- Topological analysis for accessibility compliance

Success Criteria:

- 100% IFC model conversion to RDF without data loss
- SPARQL queries execute in <500ms for 50,000-element models
- Successful integration of GNN outputs into knowledge graph

2.3 SHACL Rule Framework (Months 9-11) Goal: Replace hard-coded rules with maintainable, version-controlled constraint shapes

Why SHACL Over Hard-Coded Rules:

SHACL Shape Development:

Dimension and Hard-Coded and SHACL
Code Update Cycle and 6-12 months and Immediate
Execution Speed and 10ms/rule and 30ms/rule
Auditability and Black box and Full transparency
Multi-Jurisdiction Support and Requires recompilation and Add new shape file

Table 3: SHACL advantages for compliance validation[6]

Create constraint shapes for:

1. IBC 2024 Fire Safety (45 shapes covering exit paths, fire ratings, clearances)
2. ADA Accessibility (32 shapes for doorways, slopes, turning radii)
3. NFPA 70 Electrical (28 shapes for panel clearances, working spaces)
4. ISO 19650 Information Requirements (18 shapes for naming conventions, classification codes)

Example SHACL Constraint:

```
ex:FireExitDoorShape a sh:NodeShape ; sh:targetClass ex:FireExit ; sh:property
[ sh:path ex:width ; sh:minInclusive 1.07 ; # 42 inches per IBC 1003 sh:message
"Fire exit width must be ≥1.07m (42 inches)" ; sh:severity sh:Violation ;] ;
sh:property [ sh:path ex:clearanceZone ; sh:minInclusive 3.0 ; # 3 meters
required sh:message "Fire exit requires 3m clearance zone" ;] .
```

Implementation:

- Store shapes in version-controlled TTL files (one per jurisdiction/code)
- Deploy SHACL validation via Apache Jena SHACL API
- Enable dynamic shape selection based on project location and code year
- Generate violation reports with rule citations and confidence scores

Success Criteria:

- 120+ SHACL shapes covering Phase 1 hard-coded rules
- Validation performance <100ms per shape suite
- Zero false positives on expert-annotated test models
- Audit trail: every violation linked to specific shape constraint and source code clause

2.4 Hybrid Rule System Integration (Months 10-12) Goal: Combine deterministic, GNN-assisted, and SHACL validation into unified pipeline

Architecture:

1. ****Deterministic Pre-Filter**** (60
 - Instant evaluation of unambiguous rules (door has fire rating: yes/no)

- Hard-coded for speed: <10ms per check
- 2. **GNN Semantic Classification** (35)
 - Infer element function when IFC metadata incomplete
 - Confidence threshold: >0.8 auto-accept, 0.6-0.8 human review, <0.6 skip
- 3. **SHACL Constraint Validation** (all elements)
 - Apply function-appropriate shape constraints
 - Conditional rules: different requirements for fire exits vs standard doors
- 4. **Human Review Layer** (5)
 - Flag low-confidence inferences for expert verification
 - UI-based review workflow with accept/reject/reclassify options

Processing Pipeline:

IFC Model Input ↓ [Deterministic Rules] → 60% validated (instant) ↓ [GNN Classification] → Semantic enrichment (200ms/element) ↓ [RDF Graph Update] → Knowledge graph population ↓ [SHACL Validation] → Apply function-specific constraints (100ms/shape) ↓ [Geometric Clearance] → Phase 3 component (placeholder) ↓ Compliance Report (BCF + Audit Trail)

Success Criteria:

- End-to-end validation in <2 minutes for 50,000-element model
- 95% automation rate (5% requiring human review)
- Precision >90%, Recall >85% against expert-annotated benchmarks

2.5 Web Application Development (Months 10-12) Goal: Replace CLI with interactive web interface for broader accessibility

Features:

- IFC model upload with drag-and-drop (max 500MB file size)
- Project configuration: jurisdiction selection, code year, custom rule sets
- Real-time validation progress monitoring
- Interactive violation browser with 3D element highlighting
- Confidence score display for GNN inferences
- Approval workflow for ambiguous classifications
- BCF export and PDF report generation
- User authentication and project workspace management

Technology Stack:

- Frontend: React.js with Three.js for 3D visualization
- Backend: FastAPI (Python) for validation orchestration
- Database: PostgreSQL for projects, MongoDB for IFC metadata
- Triple Store: Apache Jena Fuseki for RDF/SPARQL
- Queue: Celery + Redis for async validation jobs

Success Criteria:

- Support 10 concurrent validation jobs
- Web interface responsive on desktop and tablet
- 3D model viewer renders 100,000-element models smoothly
- User onboarding documentation and video tutorials

Phase 2 Milestones

Month	Milestone	Deliverable
9	GNN Model Trained	85%+ accuracy on test set
10	RDF Pipeline Complete	Full IFC-to-RDF conversion
11	SHACL Shapes Deployed	120+ constraint shapes validated
12	Web Application Launch	Beta release for pilot users

Phase 3: Geometric Precision and Performance (Months 13-18)

Objective

Implement advanced computational geometry for precise “soft clash” detection of invisible clearance zones, addressing the core gap in existing coordination tools.

Key Deliverables

3.1 Minkowski Sum Clearance Engine (Months 13-15) Goal: Generate precise 3D clearance volumes around maintainable equipment and validate spatial compliance

Mathematical Foundation:

The “halo effect” is computed via Minkowski sum[7]:

$$P_{offset} = P \oplus B_r$$

Where:

- P = building element geometry
- B_r = sphere of radius r (clearance requirement)
- P_{offset} = clearance volume around element

Implementation Approach:

1. **Geometry Extraction**
 - Extract IFC element geometry as triangulated mesh
 - Support for IfcBRep, IfcSweptSolid, IfcCSG representations
 - Convert to unified mesh format (OBJ or STL)
2. **Two-Phase Spatial Indexing**
 - Phase 1: AABB (Axis-Aligned Bounding Box) pre-filter
 - Cost: <1ms per element pair
 - Eliminates 95%+ of non-intersecting pairs
 - Phase 2: Exact Minkowski sum computation
 - Applied only when bounding boxes overlap
 - Cost: 100-500ms per element (parallelizable)
3. **Clearance Volume Storage**
 - Store offset surfaces as new IfcSpatialElement entities
 - Index in spatial R-tree for efficient querying
 - Link to source element and governing rule
4. **Intersection Testing**
 - Query all elements within clearance zone
 - Triangle-triangle intersection tests for exact validation
 - Report gap measurements and violating elements

Example Validation Workflow:

Electrical Panel (NFPA 70 requirement): — Required clearance: 1m width × 0.9m depth × 2m height — Generate Minkowski sum of panel + clearance box — Query spatial index for elements within clearance zone — Detect: Storage cabinet intrudes 0.15m into clearance — Violation: "Panel working space obstructed (gap: 0.75m, required: 0.9m)"

Key Technologies:

- CGAL (Computational Geometry Algorithms Library) for Minkowski sum
- SciPy spatial module for R-tree indexing

- NumPy for geometric computations
- GPU acceleration via CUDA for large models (optional Phase 4 enhancement)

Success Criteria:

- Compute clearance zones for 1,000 equipment elements in <5 minutes
- Detect soft clashes with 100% accuracy (validated against manual measurements)
- Processing time <500ms per element on 8-core CPU
- Generate 3D clearance visualizations for BCF viewpoints

3.2 GeoSPARQL Integration (Months 14-16) Goal: Enable semantic spatial queries combining geometric reasoning with RDF knowledge

Capabilities:

- Spatial relationship queries: within, intersects, adjacent, near
- Path analysis for egress route validation
- Distance calculations between classified elements
- Topological reasoning for accessibility compliance

Example Queries:

Fire Exit Path Validation: `SELECT ?door ?corridor ?stairwell WHERE { ?door rdf:type ex:FireExit ; geo:sfWithin ?corridor . ?corridor rdf:type ex:EgressCorridor ; ex:connectsTo ?stairwell . ?stairwell rdf:type ex:EvacuationStairwell ; ex:exitsTo ?exterior . FILTER NOT EXISTS { ?obstacle geo:sfIntersects ?corridor . ?obstacle ex:obstructsEgress true . } }`

ADA Accessible Route Analysis: `SELECT ?route (MIN(?slope) AS ?maxSlope) (MIN(?width) AS ?minWidth) WHERE { ?route rdf:type ex:AccessibleRoute ; ex:slopePercent ?slope ; ex:clearWidth ?width . FILTER(?slope < 5.0 && ?width >= 0.91) } GROUP BY ?route HAVING (COUNT(?route) > 0)`

Implementation:

- Deploy Apache Jena with GeoSPARQL extension
- Convert clearance volumes to WKT (Well-Known Text) geometry format
- Index geometric literals in spatial database
- Integrate spatial query results into validation pipeline

Success Criteria:

- Execute spatial queries in <1 second for 100,000-element models
- Validate egress paths from any point to building exit
- Detect accessibility violations based on spatial topology

3.3 Multi-Jurisdiction Rule Composition (Months 15-17) Goal: Support simultaneous validation against multiple building codes with conflict resolution

Challenge:

Buildings must comply with:

- International codes (IBC, IFC)
- National codes (country-specific)
- Local amendments (city/municipality)
- Project-specific requirements (BEP, owner standards)

Conflicts may arise (e.g., IBC requires 42-inch exit, local code requires 48-inch).

Solution Architecture:

1. **Rule Package System**
 - Modular SHACL shape sets per jurisdiction
 - Version control per code year (IBC2021, IBC2024)
 - Inheritance hierarchy: International → National → Local
2. **Conflict Detection**
 - Parse all active shape constraints
 - Identify overlapping properties with different thresholds
 - Flag conflicts for user resolution
3. **Resolution Strategies**
 - Most stringent rule wins (default)
 - Explicit precedence ordering (local > national > international)
 - User-defined overrides for project-specific exceptions
4. **Audit Trail**
 - Track which jurisdiction's rule triggered each violation
 - Document conflict resolution decisions
 - Link to source code clauses in PDF regulations

Example Rule Package:

```
ex:ChicagoProjectRules ex:inheritsFrom ex:IBC2024, ex:IllinoisBuildingCode,
ex:ChicagoAmendments; ex:appliesTo "Commercial High-Rise"; ex:conflictResolution
ex:MostStringent; ex:customExceptions [ ex:rule ex:FireExitWidth; ex:override
"1.22"^^xsd:decimal; # 48 inches ex:justification "Owner requirement per
project BEP Section 3.2"; ] .
```

Success Criteria:

Model Size & Phase 2 Performance & Phase 3 Target
10,000 elements & 2 minutes & 30 seconds
50,000 elements & 12 minutes & 2 minutes
100,000 elements & 35 minutes & 5 minutes
500,000 elements & N/A & 20 minutes

Table 5: Validation performance improvement targets

- Support 10+ jurisdiction rule packages simultaneously
- Detect and report conflicting requirements
- Generate jurisdiction-specific compliance reports
- Validation time increase <20% with multiple rule sets active

3.4 Performance Optimization (Months 16-18) Goal: Scale validation to enterprise-level federated models (500,000+ elements)

Optimization Strategies:

1. **Parallel Processing**
 - Distribute element validation across 16+ CPU cores
 - GPU acceleration for Minkowski sum computations
 - Asynchronous SHACL validation per shape
2. **Incremental Validation**
 - Track model changes between validation runs
 - Re-validate only modified elements and affected neighbors
 - Cache validation results for unchanged elements
3. **Spatial Indexing**
 - R-tree spatial index for $O(\log n)$ collision queries
 - Octree decomposition for large building volumes
 - Level-of-detail (LOD) for distant elements
4. **Database Optimization**
 - Triple store sharding for RDF graphs >10M triples
 - Materialized SPARQL views for common queries
 - Connection pooling and query caching

Performance Targets:

Success Criteria:

- Validate 500,000-element federated model in <20 minutes

- Support real-time validation (<5 seconds) for incremental model changes
- Memory footprint <16GB for largest models
- 95%+ CPU utilization during validation (efficient parallelization)

3.5 Advanced BCF Reporting (Months 17-18) Goal: Enhance compliance reports with geometric visualizations and remediation guidance

New Features:

- 3D visualization of clearance zones in BCF viewpoints
- Color-coded violation severity (critical/major/minor)
- Gap measurements with dimension annotations
- Before/after comparison views for resolved issues
- AI-generated remediation suggestions
- Multi-discipline issue routing (architect/MEP/structural)
- Integration with project management tools (Procore, BIM 360)

Remediation Suggestion Engine:

```
def suggest_remediation(violation): if violation.type == "ClearanceViolation": gap_shortfall = violation.required - violation.actual return { "options": [ f"Relocate {violation.element} by {gap_shortfall}m", f"Reduce {violation.element} size by {gap_shortfall}m", f"Reconfigure adjacent elements to create clearance"], "impact_analysis": estimate_downstream_changes(violation), "cost_estimate": rough_order_magnitude(violation) }
```

Success Criteria:

- BCF reports include 3D clearance visualizations
- Remediation suggestions provided for 80%+ of violations
- Issue routing based on element discipline classification
- Export compatibility with BIM 360, Procore, Autodesk Construction Cloud

Phase 3 Milestones

Month	Milestone	Deliverable
15	Minkowski Sum Engine	Precise soft clash detection operational
16	GeoSPARQL Deployed	Spatial queries integrated into validation
17	Multi-Jurisdiction Support	10+ rule packages with conflict resolution
18	Performance Optimized	500K-element validation in <20 minutes

Phase 4: Enterprise Scale & Advanced Features (Months 19-24)

Objective

Transform BIMGUARD AI into production-grade enterprise platform with temporal validation, custom ontologies, and third-party integration ecosystem.

Key Deliverables

4.1 4D Temporal Compliance (Months 19-21) Goal: Validate compliance across construction phases, recognizing temporary conditions

Problem Statement:

Current validation assumes final building state. During construction:

- Clearance zones may be temporarily obstructed
- Egress paths may be incomplete
- Equipment may be staged in future maintenance zones

Solution: Time-Aware Validation

1. **Construction Phase Modeling**
 - Extend IFC model with construction sequence metadata
 - Define validation rules per construction phase
 - Track "acceptable temporary conditions" vs permanent violations
2. **4D BIM Integration**
 - Import 4D schedules from Navisworks TimeLiner, Synchro, MS Project
 - Link IFC elements to construction activities
 - Generate phase-specific compliance snapshots
3. **Temporal Rule Constraints**
 - SHACL shapes with time-based activation conditions
 - Exemptions for temporary construction configurations
 - Progressive compliance tracking toward final condition

Example Temporal Rule:

```
ex:ElectricalPanelClearance sh:property [ sh:path ex:clearanceZone ; sh:minInclusive 0.9 ; ex:temporalCondition [ ex:activeDuring "Construction Phase 3-Final" ; ex:exemptDuring "Construction Phase 1-2" ; ex:reason "Panel energization in Phase 3" ;] ; ] .
```

Validation Workflow:

1. User selects construction phase date

2. System filters active rules for that phase
3. Validation highlights:
 - Current compliance status
 - Future compliance risks (Phase N+1)
 - Temporary exemptions expiring soon
4. Report: "67 temporary violations, 12 must resolve before Phase 3"

Success Criteria:

- Import 4D schedules from 3 major platforms (Navisworks, Synchro, Primavera)
- Phase-specific validation reports
- Temporal violation tracking over project lifecycle
- Predictive alerts for future compliance risks

4.2 Custom Ontology Builder (Months 20-22) Goal: Enable clients to define domain-specific element classifications and rules

Use Cases:

- Healthcare facilities: medical gas systems, infection control zones
- Data centers: hot/cold aisle clearances, equipment rack spacing
- Industrial facilities: process equipment, hazardous area classifications
- Educational facilities: ADA-compliant classrooms, lab safety zones

Ontology Development Tools:

1. **Visual Ontology Editor**
 - Web-based interface for defining custom element classes
 - Drag-and-drop property assignment
 - Inheritance from base ifcOWL classes
 - Validation rule template library
2. **Rule Authoring Wizard**
 - Natural language rule input converted to SHACL
 - Example: "Medical gas outlets must be 1.5m from electrical panels"
 - LLM-assisted SHACL shape generation with human review
 - Test rule against sample models before deployment
3. **Ontology Version Control**
 - Git-based versioning for custom ontologies
 - Branching for project-specific variations

- Merge conflict resolution for collaborative editing

4. **Marketplace Integration**

- Share custom ontologies with community
- Download industry-standard ontologies (healthcare, data center)
- Version compatibility checking

Success Criteria:

- Non-technical users create custom element classes via UI
- LLM generates valid SHACL shapes from natural language rules with 90%+ accuracy
- Custom ontologies deployable without code changes
- Community marketplace with 20+ industry-specific ontologies

4.3 API & Integration Ecosystem (Months 21-23) Goal: Transform BIMGUARD AI into platform accessible by third-party tools and services

API Capabilities:

1. **RESTful Validation API**

- Endpoints: /validate, /rules, /reports, /models
- Synchronous validation for small models (<10K elements)
- Asynchronous jobs for large models with webhook callbacks
- Authentication: OAuth 2.0, API keys

2. **Streaming Validation API**

- WebSocket connection for real-time model changes
- Incremental validation as designers edit in Revit/ArchiCAD
- Sub-second feedback loop for rapid iteration

3. **Rule Management API**

- Query available rule packages and jurisdictions
- Upload custom SHACL shapes
- Version control and activation of rule sets

4. **Third-Party Integrations**

- Autodesk Revit plugin: validate on save, display violations in model
- Autodesk BIM 360: automated validation on model upload
- BlenderBIM: open-source integration for OSArch community
- Procore: link compliance reports to project tasks

SDK Development:

- Python SDK for automation scripts
- JavaScript SDK for web integrations
- .NET SDK for Revit plugin development
- Comprehensive API documentation with code examples

Success Criteria:

- API response time <500ms for synchronous validation (10K elements)
- 99.9% API uptime (enterprise SLA)
- 5 third-party integrations launched (Revit, BIM 360, BlenderBIM, Procore, +1)
- 100+ API users within 3 months of launch

4.4 Machine Learning Enhancements (Months 22-24) Goal: Leverage historical validation data to improve prediction accuracy and detect novel patterns

Advanced ML Capabilities:

1. **Graph Neural Network Improvements**
 - Expand training data to 20,000+ labeled elements
 - Multi-task learning: simultaneous classification of element type, function, and risk level
 - Active learning: prioritize uncertain predictions for human labeling
 - Target accuracy: 95
2. **Violation Prediction Models**
 - Train on historical validation reports to predict violation likelihood
 - Risk scoring per element: probability of future non-compliance
 - Proactive alerts: "This design pattern historically fails ADA compliance"
3. **Anomaly Detection**
 - Unsupervised learning to identify atypical spatial configurations
 - Flag novel patterns not covered by existing rules
 - Suggest new rules based on detected anomalies
4. **Natural Language Rule Extraction**
 - LLM fine-tuning on building code corpora
 - Automated extraction of rule parameters from PDF documents

- Accuracy target: 80

ML Operations Infrastructure:

- Model training pipeline: automated retraining on new validation data
- A/B testing framework: compare model versions in production
- Model monitoring: track prediction accuracy drift over time
- Explainability: SHAP values for GNN predictions

Success Criteria:

- GNN classification accuracy >95% on diverse building types
- Violation risk prediction with 70%+ precision at 80%+ recall
- Anomaly detection flags 15+ novel compliance patterns in pilot dataset
- LLM extracts rules from new code PDFs with 80%+ parameter accuracy

4.5 Enterprise Deployment & DevOps (Months 22-24) Goal: Production-ready infrastructure with enterprise scalability, security, and reliability

Infrastructure:

- Cloud deployment: AWS/Azure multi-region architecture
- Kubernetes orchestration for microservices
- Load balancing: support 1,000+ concurrent users
- Autoscaling: elastic compute for validation workload spikes
- Data storage: S3/Blob Storage for IFC models, PostgreSQL for metadata
- Triple store cluster: Apache Jena Fuseki with replication

Security & Compliance:

- SOC 2 Type II compliance
- GDPR compliance for EU customers
- End-to-end encryption for model uploads
- Role-based access control (RBAC)
- Audit logging for all validation activities
- ISO 27001 information security management

Monitoring & Reliability:

- 99.9
- Prometheus + Grafana for system metrics
- Distributed tracing (Jaeger) for request debugging

- Automated alerting for performance degradation
- Disaster recovery: daily backups, 4-hour RTO

DevOps Practices:

- CI/CD pipeline: automated testing and deployment
- Infrastructure as Code (Terraform)
- Blue-green deployments for zero-downtime updates
- Feature flags for gradual rollout
- Automated security scanning (Snyk, Dependabot)

Success Criteria:

- Support 1,000 concurrent validation jobs
- 99.9% monthly uptime
- API p95 latency <500ms
- Pass SOC 2 audit
- Zero data breaches

Phase 4 Milestones

Month	Milestone	Deliverable
21	4D Validation	Construction phase compliance tracking
22	Custom Ontology Builder	UI for client-defined rules
23	API Platform Launch	5 third-party integrations live
24	Enterprise Deployment	Production infrastructure with 99.9% SLA

Technology Stack Summary

Core Technologies

Open Source Commitments

- Core validation algorithms: Open source (MIT license)
- SHACL shape libraries: Public GitHub repository
- IFC test models: Contribution to buildingSMART samples
- Community: Active participation in OSArch and buildingSMART forums

Layer & Technologies
IFC Parsing & IfcOpenShell, ifcopenshell-python Data Processing & pandas, NumPy, SciPy Machine Learning & PyTorch, PyTorch Geometric, scikit-learn Semantic Web & Apache Jena, rdflib, pyshacl Computational Geometry & CGAL, Shapely, trimesh Spatial Indexing & R-tree, Octree, GeoSPARQL Backend & FastAPI, Celery, Redis, PostgreSQL, MongoDB Frontend & React.js, Three.js, Babylon.js DevOps & Docker, Kubernetes, Terraform, GitHub Actions Cloud & AWS/Azure (compute, storage, databases)

Table 8: Complete technology stack for BIMGUARD AI

Risk & Mitigation & Contingency
GNN accuracy below target (85%) & Transfer learning from pre-trained models; expand training dataset & F SHACL performance bottleneck & Triple store optimization; query caching; materialized views & Hybrid syst Minkowski sum computation too slow & GPU acceleration; spatial indexing; LOD for distant elements & App IFC parsing failures & Extensive testing with buildingSMART samples; fallback to IFC2x3 & Manual geometry

Table 9: Technical risk mitigation strategies

Risk Management

Technical Risks

Market Risks

- **Competition from established tools (Solibri, BIM 360):** Differentiate through soft clash detection and open standards approach
- **User adoption of new workflows:** Pilot program with early adopters; comprehensive training materials
- **Code version fragmentation:** Modular rule packages per jurisdiction; community contributions
- **Proprietary BIM tool lock-in:** Emphasize openBIM and vendor-neutral compliance

Operational Risks

- **Team expertise gaps:** Hire specialists in computational geometry and semantic web technologies
- **Infrastructure costs:** Start with pay-per-use cloud model; optimize before scaling

- ****Data privacy concerns:**** SOC 2 compliance from day one; clear data handling policies
 - ****Open source community engagement:**** Dedicate resources to community building and support
-

Success Metrics & KPIs

Phase 1 (Foundation)

- IFC parsing success rate: >95%
- Rule evaluation accuracy: 100% on deterministic checks
- BCF import success rate: 100% in Revit

Phase 2 (Intelligence)

- GNN classification accuracy: >85%
- SHACL validation coverage: 120+ shapes
- Web application user satisfaction: >4.0/5.0

Phase 3 (Precision)

- Soft clash detection accuracy: 100%
- 500K-element validation time: <20 minutes
- Multi-jurisdiction conflict detection: 100% of known conflicts

Phase 4 (Enterprise)

- API uptime: 99.9%
 - Third-party integrations: 5+
 - Custom ontologies created: 20+ (community)
 - Enterprise customers: 10+
-

Budget & Resource Allocation

Team Structure

Infrastructure Costs (Monthly, Phase 3-4)

- Cloud compute (AWS/Azure): \$5,000-\$10,000
- Database hosting: \$2,000-\$4,000
- Storage (models, backups): \$1,000-\$2,000
- CDN & networking: \$500-\$1,000

Role & Phase 1-2 & Phase 3-4
Project Lead & 1 FTE & 1 FTE
ML Engineers & 1 FTE & 2 FTE
Backend Developers & 2 FTE & 3 FTE
Frontend Developers & 1 FTE & 2 FTE
DevOps Engineers & 0.5 FTE & 1 FTE
BIM Domain Experts & 1 FTE & 1 FTE
QA/Testing & 0.5 FTE & 1 FTE
Technical Writer & 0.5 FTE & 1 FTE
Total & 7.5 FTE & 12 FTE

Table 10: Recommended team composition

Feature & Solibri & BIMGUARD AI
Soft clash detection & Limited & ✓ Full 3D clearances
AI semantic classification & X & ✓ GNN-based
Open rule format & X Proprietary & ✓ SHACL (W3C standard)
Custom ontologies & X & ✓ Visual editor
4D temporal validation & X & ✓ Phase-aware
Pricing model & Perpetual license & SaaS subscription

Table 11: Competitive comparison with market leader

- Third-party APIs (LLMs): \$2,000-\$5,000
- Monitoring & logging: \$500-\$1,000

Total monthly infrastructure: \$11,000-\$23,000 (scales with usage)

Competitive Differentiation

vs. Solibri Model Checker

vs. Autodesk BIM 360

- **BIM 360:** Cloud collaboration platform with basic rule checking
- **BIMGUARD AI:** Specialized compliance engine with advanced spatial reasoning
- **Integration strategy:** Complement BIM 360 via API integration

Value Proposition

"BIMGUARD AI is the only compliance platform that combines AI-driven semantic understanding with precise geometric validation using open standards, en-

abling architects and engineers to detect invisible spatial violations that traditional coordination tools miss.”

References

- [1] Gallaher, M. P., O'Connor, A. C., Dettbarn, J. L., & Gilday, L. T. (2004). Cost Analysis of Inadequate Interoperability in the U.S. Capital Facilities Industry (NIST GCR 04-867). National Institute of Standards and Technology.
- [2] buildingSMART International. (2020). IFC4 Reference View. <https://standards.buildingsmart.org/MVD/RELEASE>
- [3] NFPA. (2023). NFPA 70: National Electrical Code. National Fire Protection Association.
- [4] Research shows Edge-Conditioned Graph Neural Networks achieve 91.8% accuracy at room classification with contextual awareness through neighborhood relationships.
- [5] buildingSMART. (2020). ifcOWL Ontology for Industry Foundation Classes. <https://technical.buildingsmart.org/standards/ifc/ifc-formats/ifcowl/>
- [6] W3C. (2017). Shapes Constraint Language (SHACL). <https://www.w3.org/TR/shacl/>
- [7] Minkowski sum mathematical foundation for clearance volume computation: $P_{offset} = P \oplus B_r$ where P is element geometry and B_r is clearance sphere.
- [8] ISO. (2018). ISO 19650-1:2018 — Organisation and digitisation of information about buildings and civil engineering works. International Organization for Standardization.
- [9] Eastman, C., Lee, J., Jeong, Y., & Lee, J. (2009). Automatic rule-based checking of building designs. *Automation in Construction*, 18(8), 1011–1033.
- [10] buildingSMART International. (2021). BCF (BIM Collaboration Format) Version 2.1. <https://github.com/buildingSMART/BCF-XML>